

Cheat sheet SQL completo

Versione finale aggiornata con EXTRACT, che serve a estrarre parti specifiche da date e timestamp come anno, mese e giorno.

Clausola	Operazione	A cosa serve	Esempio
CTE	WITH	Definisce un result set temporaneo con nome, usabile nella query principale	WITH clienti_attivi AS (SELECT * FROM clienti WHERE età >= 18)
CTE	WITH multipla	Permette di definire più CTE separate da virgola	WITH cte1 AS (...), cte2 AS (...) SELECT * FROM cte1 JOIN cte2 ON ...;
SELECT	DISTINCT	Mostra solo valori unici	SELECT DISTINCT città FROM clienti;
SELECT	COUNT()	Conta le righe o i valori non nulli	SELECT COUNT(*) FROM clienti;
SELECT	SUM()	Somma i valori numerici	SELECT SUM(importo) FROM ordini;
SELECT	AVG()	Calcola la media	SELECT AVG(prezzo) FROM prodotti;
SELECT	MIN()	Trova il valore minimo	SELECT MIN(prezzo) FROM prodotti;
SELECT	MAX()	Trova il valore massimo	SELECT MAX(prezzo) FROM prodotti;
SELECT	LENGTH()	Restituisce la lunghezza di una stringa	SELECT LENGTH(nome) FROM clienti;
SELECT	LEFT()	Estrae i primi N caratteri da sinistra	SELECT LEFT(nome, 3) FROM clienti;
SELECT	RIGHT()	Estrae gli ultimi N caratteri da destra	SELECT RIGHT(telefono, 3) FROM clienti;
SELECT	SUBSTRING()	Estrae una parte della stringa da una posizione data	SELECT SUBSTRING(nome, 1, 3) FROM clienti;
SELECT	POSITION()	Restituisce la posizione di una sottostringa	SELECT POSITION("@" IN email) FROM clienti;
SELECT	UPPER()	Trasforma il testo in maiuscolo	SELECT UPPER(nome) FROM clienti;
SELECT	LOWER()	Trasforma il testo in minuscolo	SELECT LOWER(email) FROM clienti;
SELECT	CONCAT()	Unisce più stringhe in una sola	SELECT CONCAT(nome, " ", cognome) FROM clienti;
SELECT	CONCAT_WS()	Unisce più stringhe inserendo un separatore	SELECT CONCAT_WS(" ", nome, cognome) FROM clienti;
SELECT	CAST()	Converte un valore in un altro tipo	SELECT CAST(prezzo AS INT) FROM prodotti;
SELECT	COALESCE()	Restituisce il primo valore non NULL	SELECT COALESCE(telefono, email, "N/D") FROM clienti;
SELECT	EXTRACT()	Estrae una parte da una data o timestamp	SELECT EXTRACT(MONTH FROM data_ordine) FROM ordini;
SELECT	CASE WHEN	Applica logica condizionale	SELECT CASE WHEN prezzo > 100 THEN "Alto" ELSE "Basso" END FROM prodotti;
SELECT	AS	Rinomina una colonna nel risultato	SELECT nome AS nome_cliente FROM clienti;
WINDOW	ROW_NUMBER() OVER(...)	Numera le righe in sequenza dentro una finestra	ROW_NUMBER() OVER (ORDER BY importo DESC) AS rn
WINDOW	PARTITION BY	Divide il risultato in partizioni per calcoli indipendenti	ROW_NUMBER() OVER (PARTITION BY cliente_id ORDER BY importo DESC) AS rn
WINDOW	ORDER BY in OVER()	Definisce l'ordine interno della finestra	SUM(importo) OVER (PARTITION BY cliente_id ORDER BY data_ordine)
FROM	tabella	Definisce la tabella di partenza	FROM clienti
FROM	alias	Assegna un nome breve alla tabella	FROM clienti c
FROM	subquery	Usa una query come tabella temporanea	FROM (SELECT * FROM clienti) c
JOIN	INNER JOIN	Tiene solo le righe con corrispondenza in entrambe	INNER JOIN ordini o ON c.id = o.cliente_id
JOIN	LEFT JOIN	Tiene tutte le righe della tabella sinistra e le corrispondenze della destra	LEFT JOIN ordini o ON c.id = o.cliente_id
JOIN	RIGHT JOIN	Tiene tutte le righe della tabella destra e le corrispondenze della sinistra	RIGHT JOIN ordini o ON c.id = o.cliente_id
JOIN	FULL OUTER JOIN	Tiene tutte le righe di entrambe	FULL OUTER JOIN ordini o ON c.id = o.cliente_id
JOIN	CROSS JOIN	Genera tutte le combinazioni possibili	CROSS JOIN categorie
JOIN	ON	Definisce la condizione di collegamento tra tabelle	ON c.id = o.cliente_id
JOIN	USING()	Collega usando una colonna con lo stesso nome	JOIN ordini USING(cliente_id)
WHERE	=	Filtra i valori uguali	WHERE città = "Roma"
WHERE	<>	Filtra i valori diversi	WHERE città <> "Roma"
WHERE	>	Filtra i valori maggiori	WHERE prezzo > 100
WHERE	<	Filtra i valori minori	WHERE prezzo < 100

Clausola	Operazione	A cosa serve	Esempio
WHERE	>=	Filtra i valori maggiori o uguali	WHERE età >= 18
WHERE	<=	Filtra i valori minori o uguali	WHERE età <= 65
WHERE	AND	Richiede che più condizioni siano vere	WHERE città = "Roma" AND età >= 18
WHERE	OR	Richiede che almeno una condizione sia vera	WHERE città = "Roma" OR città = "Milano"
WHERE	NOT	Nega una condizione	WHERE NOT città = "Roma"
WHERE	IN (...)	Controlla se un valore è in una lista	WHERE città IN ("Roma","Milano","Napoli")
WHERE	BETWEEN ... AND ...	Controlla se un valore è in un intervallo	WHERE prezzo BETWEEN 10 AND 50
WHERE	LIKE	Cerca un pattern in una stringa	WHERE nome LIKE "A%"
WHERE	IS NULL	Trova i valori nulli	WHERE telefono IS NULL
WHERE	IS NOT NULL	Trova i valori non nulli	WHERE email IS NOT NULL
WHERE	EXISTS (...)	Verifica se una sottoquery restituisce righe	WHERE EXISTS (SELECT 1 FROM ordini o WHERE o.cliente_id = c.id)
GROUP BY	una colonna	Raggruppa per una colonna	GROUP BY città
GROUP BY	più colonne	Raggruppa per più colonne	GROUP BY città, categoria
HAVING	COUNT()	Filtra gruppi in base al conteggio	HAVING COUNT(*) > 5
HAVING	SUM()	Filtra gruppi in base alla somma	HAVING SUM(importo) > 1000
HAVING	AVG()	Filtra gruppi in base alla media	HAVING AVG(prezzo) < 50
ORDER BY	ASC	Ordina in modo crescente	ORDER BY nome ASC
ORDER BY	DESC	Ordina in modo decrescente	ORDER BY prezzo DESC
ORDER BY	più colonne	Ordina usando più campi	ORDER BY città ASC, nome DESC
LIMIT	LIMIT n	Mostra solo le prime n righe	LIMIT 10
LIMIT	OFFSET n	Salta le prime n righe	LIMIT 10 OFFSET 20
UNION	UNION	Unisce due risultati senza duplicati	SELECT nome FROM clienti UNION SELECT nome FROM fornitori;
UNION	UNION ALL	Unisce due risultati con duplicati	SELECT nome FROM clienti UNION ALL SELECT nome FROM fornitori;
INSERT	VALUES	Inserisce una nuova riga	INSERT INTO clienti (nome, città) VALUES ("Mario", "Roma");
INSERT	INSERT ... SELECT	Inserisce righe prese da una query	INSERT INTO archivio_clienti SELECT * FROM clienti;
UPDATE	SET	Aggiorna una o più colonne	UPDATE clienti SET città = "Milano" WHERE id = 1;
DELETE	WHERE	Elimina le righe filtrate	DELETE FROM clienti WHERE età < 18;
CREATE	CREATE TABLE	Crea una nuova tabella	CREATE TABLE clienti (id INT, nome TEXT);
ALTER	ADD COLUMN	Aggiunge una colonna	ALTER TABLE clienti ADD email TEXT;
ALTER	RENAME	Rinomina tabella o colonna	ALTER TABLE clienti RENAME TO clienti_attivi;
DROP	DROP TABLE	Elimina una tabella	DROP TABLE clienti;

Query master

```
WITH clienti_filtrati AS (
    SELECT
        id, nome, cognome, città, età, email, telefono, indirizzo
    FROM clienti
    WHERE età >= 18
        AND città IN ('Roma', 'Milano', 'Napoli')
),
ordini_base AS (
    SELECT
        o.id, o.cliente_id, o.importo, o.data_ordine,
        ROW_NUMBER() OVER (
            PARTITION BY o.cliente_id
            ORDER BY o.importo DESC
        ) AS rn_importo
    FROM ordini o
    WHERE o.importo > 10
)
SELECT
    c.id AS cliente_id,
    UPPER(c.nome) AS nome_upper,
    LOWER(c.cognome) AS cognome_lower,
    CONCAT(c.nome, ' ', c.cognome) AS nome_completo,
    CONCAT_WS(' - ', c.nome, c.cognome, c.città) AS nome_cognome_città,
    LEFT(c.nome, 3) AS iniziale_nome,
    RIGHT(c.telefono, 3) AS ultime_3_cifre,
    SUBSTRING(c.email, 1, 5) AS email_inizio,
    POSITION('@' IN c.email) AS posizione_at,
    LENGTH(c.email) AS lunghezza_email,
    EXTRACT(MONTH FROM o.data_ordine) AS mese_ordine,
    EXTRACT(YEAR FROM o.data_ordine) AS anno_ordine,
    CAST(o.importo AS INT) AS importo_intero,
    COALESCE(c.telefono, c.email, 'N/D') AS contatto,
    CASE
        WHEN o.importo > 1000 THEN 'Alto'
        WHEN o.importo > 500 THEN 'Medio'
        ELSE 'Basso'
    END AS fascia_importo,
    o.rn_importo,
    COUNT(*) AS totale_righe,
    COUNT(DISTINCT o.id) AS ordini_unici,
    SUM(o.importo) AS fatturato_totale,
    AVG(o.importo) AS importo_medio,
    MIN(o.data_ordine) AS primo_ordine,
    MAX(o.importo) AS ordine_max
FROM clienti_filtrati c
INNER JOIN ordini_base o ON c.id = o.cliente_id
WHERE c.email IS NOT NULL
    AND c.nome LIKE 'A%'
GROUP BY
    c.id, c.nome, c.cognome, c.città, c.email, c.telefono,
    o.id, o.importo, o.data_ordine, o.rn_importo
HAVING COUNT(*) >= 1
ORDER BY fatturato_totale DESC, nome_completo ASC
LIMIT 20 OFFSET 0;
```